Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Asymmetric Cryptography

Data Privacy and Security

Data Science and Management (DASMA)

LUISS

Issues with Symmetric Cryptography

When working with symmetric cryptography, we face two main issues:

Issues with Symmetric Cryptography

When working with symmetric cryptography, we face two main issues:

1. **Key Distribution**, i.e., exchange the key securely between the two parties:
   - Without the secret shared key, users cannot use symmetric cryptography!
   - They could exchange it physically, but it is not practical for large-scale deployments.

Issues with Symmetric Cryptography

When working with symmetric cryptography, we face two main issues:

1. **Key Distribution**, i.e., exchange the key securely between the two parties:
   - Without the secret shared key, users cannot use symmetric cryptography!
   - They could exchange it physically, but it is not practical for large-scale deployments.

2. **Key Management**, i.e., when we have n users, we need to manage $n(n-1)/2$ keys:
   - There are billions of users accessing the internet, it cannot scale!
   - For instance, Amazon has over 100M users and would need to manage $100M \times 100M/2 = 50$ billion keys!
   - Even small organizations have thousands of users and may struggle to manage their keys.

Asymmetric Roles of Keys

Each user has a **pair** of keys:

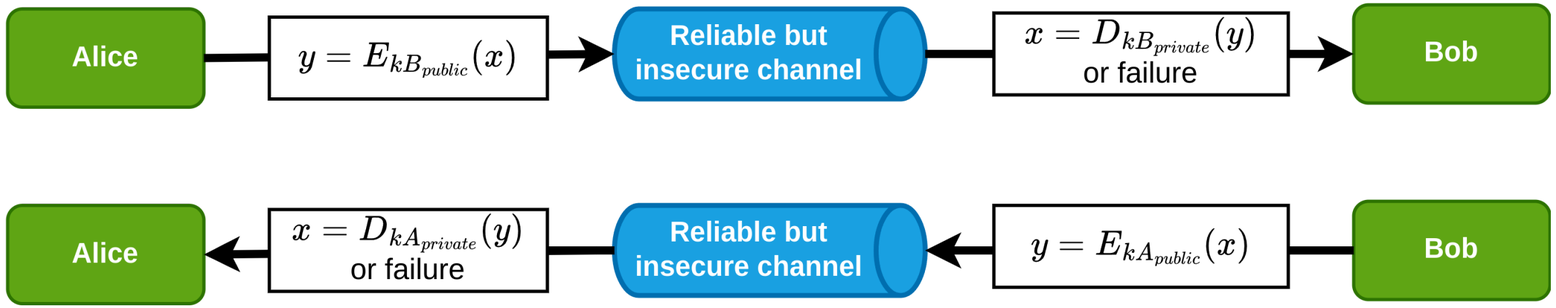
- $k_{private}$ which must be only known to the user
- k_{public} which can be known to everyone

Asymmetric Roles of Keys

Each user has a **pair** of keys:

- $k_{private}$ which must be only known to the user
- k_{public} which can be known to everyone

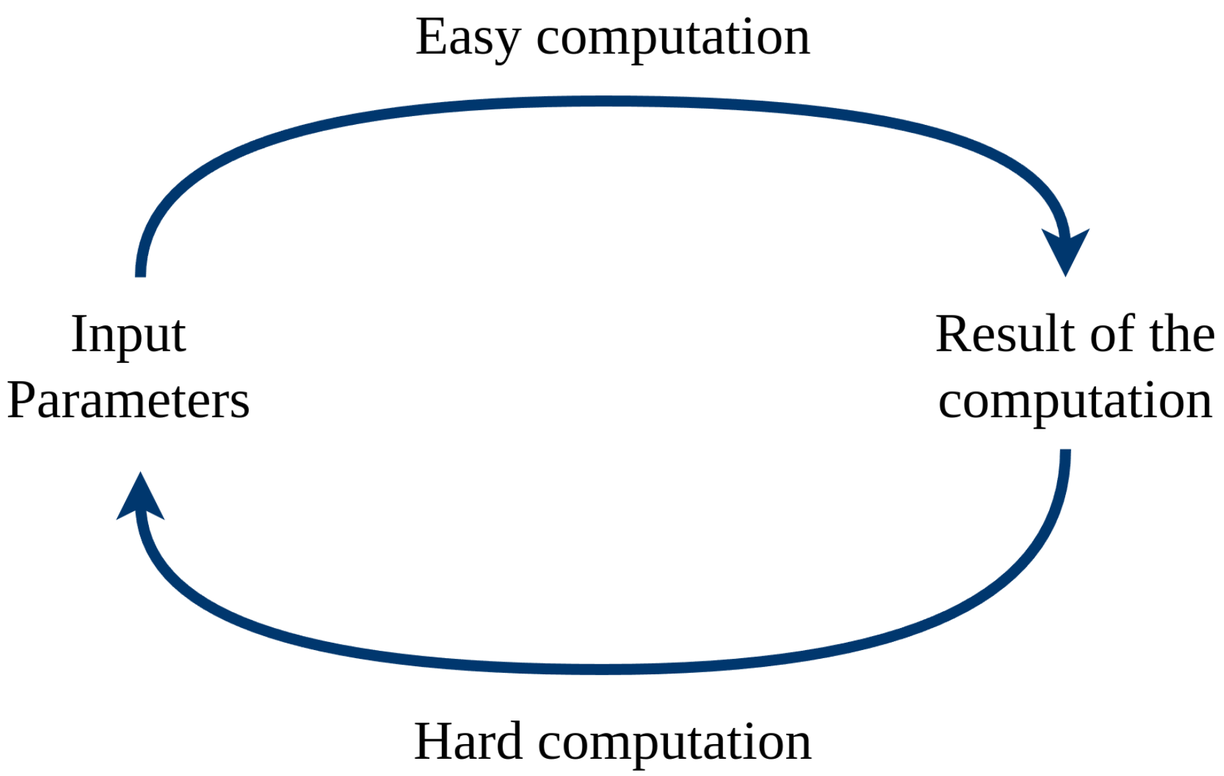
Hence:



where $kA_{private}$ is the private key of Alice and kA_{public} is the public key of Alice, similarly $kB_{private}$ is the private key of Bob and kB_{public} is the public key of Bob.

How to achieve this *asymmetry*?

Most methods exploit a mathematical problem showing this behavior:



Easy to compute the result starting from the input parameters
but hard to compute the input parameters starting from the result

Diffie and Hellman Algorithm for Key Exchange

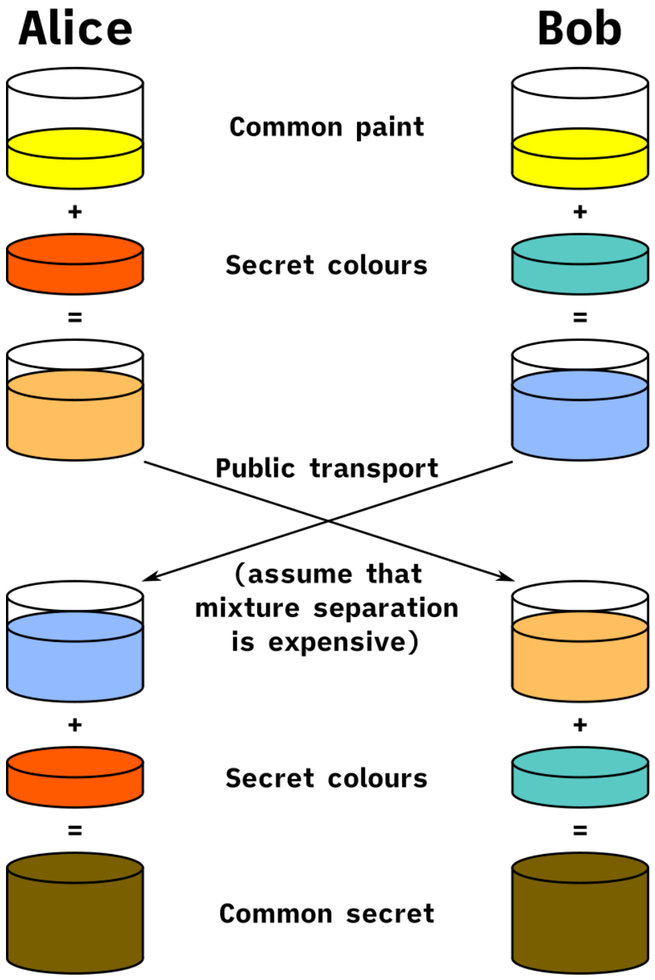
Key Exchange

Alice and Bob wants to encrypt their messages using symmetric cryptography:

- They need to exchange the symmetric key
- They don't know each other and cannot meet in person
- They cannot exchange the key over an insecure channel
- They want to agree on the key without revealing it to an eavesdropper
- They can exchange messages over an insecure channel but not the key

Intuition of Diffie-Hellman Key Exchange Algorithm

Visually*:

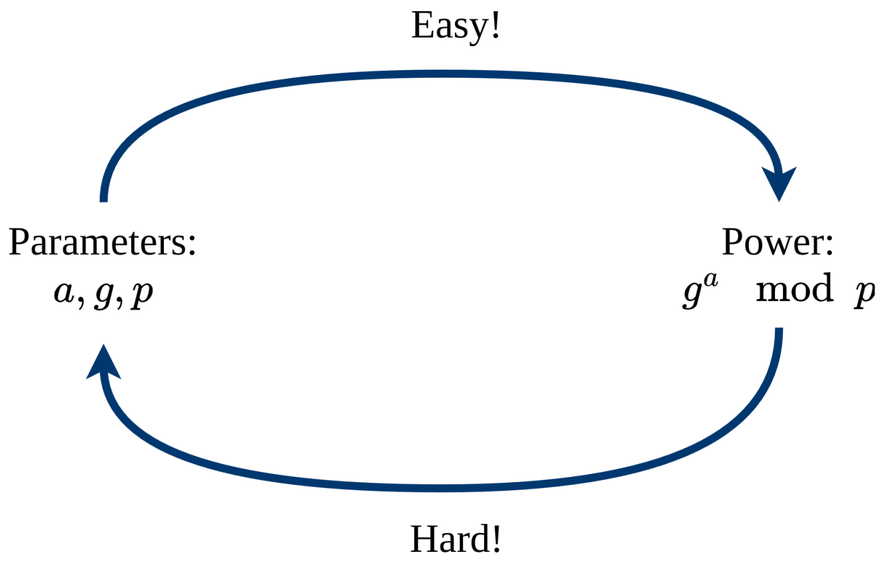


* In this example, we are assuming that mixture separation is hard to perform. It is not the case in real life.

Instead of colors we use power of numbers

Diffie-Hellman key exchange protocol exploits the Discrete Logarithm Problem (DLP):

- Given a, p, and g, it is easy to compute $g^a \mod p$
- Given the result of $g^a \mod p$, it is hard to compute a even when p and g are known.



REMARK: $x \mod p$ operation means taking the remainder of the division of x by p.

But computing the logarithm is easy!

Given a **real** number, any calculator can compute:

$$\log_g(g^a) = a \qquad \Longrightarrow \qquad \log_7(7^8) = \log_7(5764801) = 8$$

Indeed, if g is known, the larger the value of g^a, the larger the value of a will be:

- $7^0 = 1$
- $7^1 = 7$
- $7^2 = 49$
- $7^3 = 343$
- $7^4 = 2401$
- $7^5 = 16807$

The logarithm is a *monotonic* function and we can find its approximate value by hand with *some* attempts*.

* We are oversimplifying the problem to get the intuition. However, there exists numerical methods to quickly get the solution.

But moving to the discrete world changes everything

However, when we move to the **discrete** logarithm, we do not know how to do it efficiently:

- Even if g and p are known, as $g^a \mod p$ increases, a does **not** necessarily increase.

- Assuming $g = 7, p = 97$, then:
  - $7^0 \mod 97 = 1 \mod 97 = 1$
  - $7^1 \mod 97 = 7 \mod 97 = 7$
  - $7^2 \mod 97 = 49 \mod 97 = 49$
  - $7^3 \mod 97 = 343 \mod 97 = 52$
  - $7^4 \mod 97 = 2401 \mod 97 = 73$
  - $7^5 \mod 97 = 16807 \mod 97 = 26$
  - $7^6 \mod 97 = 117649 \mod 97 = 85$

- The naive approach to solve DLP is by attempting all possible values of x and checking if $g^x \equiv h \mod p$. There are some smarter approaches but they do not scale well when the input parameters are large and chosen carefully.

Diffie-Hellman Key Exchange Algorithm (1976)

Public parameters:

- Prime number p
- Generator g

Diffie-Hellman Key Exchange Algorithm (1976)

Public parameters:

- Prime number p
- Generator g

Alice:

$$a \in [1, p-1]$$
$$g^a \mod p$$

Bob:

$$b \in [1, p-1]$$
$$g^b \mod p$$

Diffie-Hellman Key Exchange Algorithm (1976)

Public parameters:

- Prime number p
- Generator g

Alice:	Exchanged messages:	Bob:
$a \in [1, p-1]$		$b \in [1, p-1]$
$g^a \mod p$	$\xrightarrow{g^a \mod p}$	$g^b \mod p$
	$\xleftarrow{g^b \mod p}$	

Diffie-Hellman Key Exchange Algorithm (1976)

Public parameters:

- Prime number p
- Generator g

Alice:	Exchanged messages:	Bob:
$a \in [1, p-1]$		$b \in [1, p-1]$
$g^a \mod p$		$g^b \mod p$
	$\xrightarrow{g^a \mod p}$	
	$\xleftarrow{g^b \mod p}$	
$(g^b)^a \mod p = g^{ab} \mod p$		$(g^a)^b \mod p = g^{ab} \mod p$

NOTICE: In the end, both parties have the same shared secret key $k = g^{ab} \bmod p$.

DH provides *Perfect Forward Secrecy*

If the secrets (a, b, k) of one communication session are compromised, then other (past or future) communication sessions will not be compromised.

The value of **forward secrecy** is that it protects past communications from being decrypted even if long-term keys are later compromised. As a result, the impact of a key compromise is limited: previously recorded traffic remains confidential, and only sessions established after the compromise are at risk until the key is revoked and replaced.

Is DH secure against a passive attacker?

> **NOTE**: A passive attacker can only eavesdrop on the communication.

Is DH secure against a <u>passive</u> attacker?

> **NOTE**: A passive attacker can only eavesdrop on the communication.

Passive attacker:

- from eavesdropping knows $g, p, g^a \bmod p, g^b \bmod p$
- can compute $g^{a+b} \bmod p$ which is different from $g^{ab} \bmod p$

Is DH secure against a <u>passive</u> attacker?

> **NOTE**: A passive attacker can only eavesdrop on the communication.

Passive attacker:

- from eavesdropping knows $g, p, g^a \bmod p, g^b \bmod p$
- can compute $g^{a+b} \bmod p$ which is different from $g^{ab} \bmod p$

Why it cannot compute the shared secret key?

Is DH secure against a <u>passive</u> attacker?

> **NOTE**: A passive attacker can only eavesdrop on the communication.

Passive attacker:

- from eavesdropping knows $g, p, g^a \bmod p, g^b \bmod p$
- can compute $g^{a+b} \bmod p$ which is different from $g^{ab} \bmod p$

Why it cannot compute the shared secret key?

To build the shared secret key it needs to recover a (or b). One way of breaking DH is to solve DLP, which is believed to be hard to break. However, there could be other ways of breaking DH that do not necessarily need to solve DLP. However, after several years it is still believed to be safe.

Is DH secure against an <u>active</u> attacker?

> **NOTE**: An active attacker can both eavesdrop and modify the communication.

Is DH secure against an <u>active</u> attacker?

> **NOTE**: An active attacker can both eavesdrop and modify the communication.

Man-in-the-Middle (MITM) attack:

1. Adversary fakes Bob identity and performs DH with Alice
2. Adversary fakes Alice identity and performs DH with Bob
3. Alice and Bob think that they have a common shared key, instead they are sharing a secret key with the adversary.

Is DH secure against an <u>active</u> attacker?

NOTE: An active attacker can both eavesdrop and modify the communication.

Man-in-the-Middle (MITM) attack:

1. Adversary fakes Bob identity and performs DH with Alice
2. Adversary fakes Alice identity and performs DH with Bob
3. Alice and Bob think that they have a common shared key, instead they are sharing a secret key with the adversary.

PROBLEM:
Alice and Bob do not ***authenticate*** the received public keys. We will see how to solve this problem when talking about authentication and the Public Key Infrastructure (PKI).

RSA (Rivest–Shamir–Adleman) cryptosystem

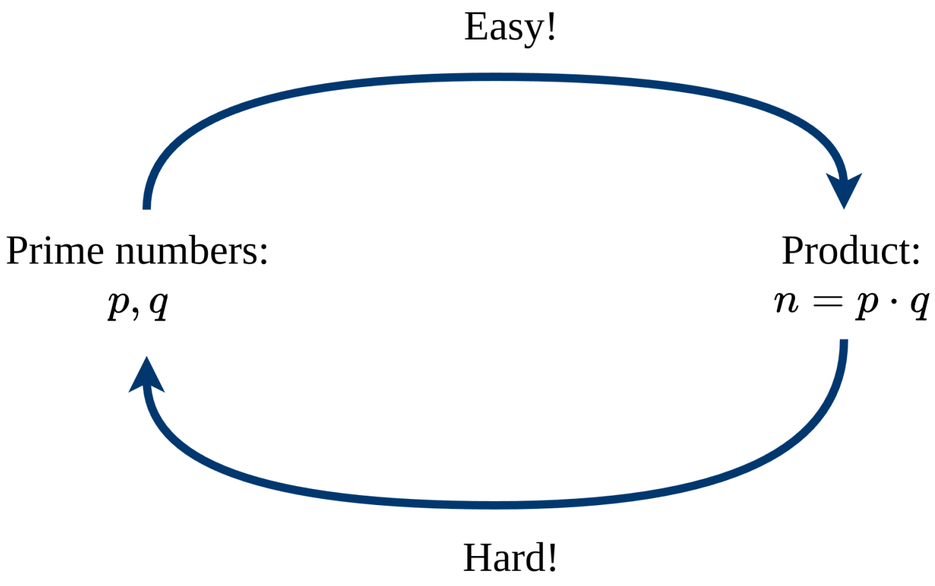
Solving the Key Management Problem

To avoid to have and distribute $n(n-1)/2$ keys, we can use asymmetric cryptography:

- each user has a public key and a private key
- the public key can be distributed to everyone, even on the internet
- the private key must be kept secret
- each user can send an encrypted message to another user using the recipient's public key

RSA

Public-key scheme proposed by Ron Rivest, Adi Shamir, and Leonard Adleman in 1977:

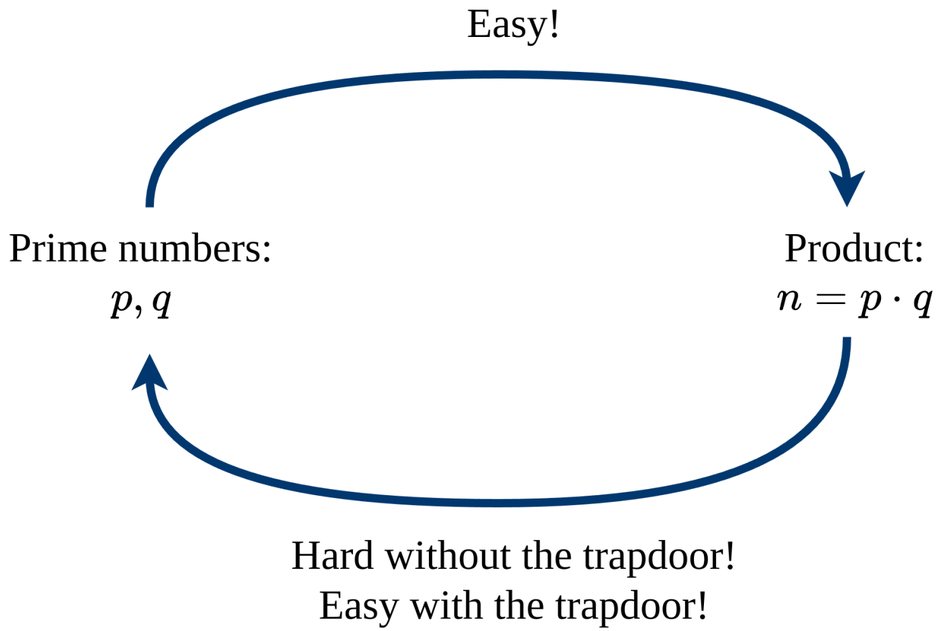


RSA is based on the ***integer factorization problem***.

* A nice and non-technical video about RSA is this **one**

Hard only when required

RSA exploits the concept of a ***trapdoor*** function:



The computation in the reverse direction is hard only when not knowing the trapdoor.

RSA Scheme

What we would like to have in principle:

- message: x
- public key: e
- private key: d (the trapdoor)
- scheme: $y = x^e$ such that $y^d = y^{ed} = x$
- work with modular arithmetic (to work easily with a finite set of numbers)

How to achieve this?

Euler's Phi Function (or Totient Function)

Euler's Phi Function (or Totient Function)

REMARK:
A number a is *coprime* to a number m when their greatest common divisor is 1, i.e. $gcd(a, m) = 1$

Euler's Phi Function (or Totient Function)

REMARK:
A number a is *coprime* to a number m when their greatest common divisor is 1, i.e. $gcd(a, m) = 1$

When considering $\mathbb{Z}_m = \{0, 1, ..., m-1\}$

How many numbers in it are coprime (relatively prime) to m, i.e., $\Phi(m)$?

A naive approach to answer the previous question is to verify:

$$gcd(a, m) \stackrel{?}{=} 1 \; \forall \, a \in \mathbb{Z}_m$$

For instance, let us consider $\mathbb{Z}_6$:

$$\begin{aligned} gcd(0,6) = 6 &\Longrightarrow \text{✗} \\ gcd(1,6) = 1 &\Longrightarrow \checkmark \\ gcd(2,6) = 2 &\Longrightarrow \text{✗} \\ gcd(3,6) = 3 &\Longrightarrow \text{✗} \\ gcd(4,6) = 2 &\Longrightarrow \text{✗} \\ gcd(5,6) = 1 &\Longrightarrow \checkmark \\ gcd(6,6) = 6 &\Longrightarrow \text{✗} \end{aligned}$$

Hence:

$$\Phi(6) = 2$$

When considering other values for m:

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
$\phi(n)$	1	1	2	2	4	2	6	4	6	4	10	4	12	6	8	8	16	6	18	8	12

Computing Efficiently the Euler's Phi Function

If we know how to factorize the number m:

$$m = p_1^{e_1} \cdot p_2^{e_2} \cdot \ldots \cdot p_k^{e_k}$$

where:

- $p_1, p_2, \ldots, p_k$ are distinct prime numbers
- $e_1, e_2, \ldots, e_k$ are positive integers

Then:

$$\Phi(m) = \prod_{i=1}^{m} (p_i^{e_i} - p_i^{e_i - 1})$$

$$m = 240$$

$$\Phi(240) = ?$$

$$m = 240$$

$$\Phi(240) = ?$$

$$m = 2^4 \cdot 3 \cdot 5 \Rightarrow \Phi(240) = (2^4 - 2^3)(3^1 - 3^0)(5^1 - 5^0) = 64$$

p is a prime number

$$\Phi(p) = ?$$

p is a prime number

$$\Phi(p) = ?$$

$$\phi(p) = (p^1 - p^0) = p - 1$$

n is the product of two primes p and q

$$\Phi(n) = \Phi(pq)\,?$$

n is the product of two primes p and q

$$\Phi(n) = \Phi(pq)\,?$$

$$\phi(n) = (p^1 - p^0)(q^1 - q^0) = (p-1)(q-1)$$

RSA Key Generation

RSA Key Generation

1. Choose two large prime numbers p and q

RSA Key Generation

1. Choose two large prime numbers p and q

2. Compute their product: $n = p \cdot q$

RSA Key Generation

1. Choose two large prime numbers p and q

2. Compute their product: $n = p \cdot q$

3. Compute the totient: $\Phi(n) = (p-1)(q-1)$

RSA Key Generation

1. Choose two large prime numbers p and q

2. Compute their product: $n = p \cdot q$

3. Compute the totient: $\Phi(n) = (p-1)(q-1)$

4. Choose an integer e such that:
   - must be prime
   - must be smaller than $\Phi(n)$, i.e., $1 < e < \phi(n)$
   - must not be a factor of $\Phi(n)$, i.e., must be coprime to $\phi(n)$
   - in practice:
     - we can use the Euclidean Algorithm to test if e is coprime to $\phi(n)$
     - most implementations use $e = 65537$ (after checking $\Phi(n)$ isn't divisible by 65537)

RSA Key Generation

1. Choose two large prime numbers p and q

2. Compute their product: $n = p \cdot q$

3. Compute the totient: $\Phi(n) = (p-1)(q-1)$

4. Choose an integer e such that:
   - must be prime
   - must be smaller than $\Phi(n)$, i.e., $1 < e < \phi(n)$
   - must not be a factor of $\Phi(n)$, i.e., must be coprime to $\phi(n)$
   - in practice:
     - we can use the Euclidean Algorithm to test if e is coprime to $\phi(n)$
     - most implementations use $e = 65537$ (after checking $\Phi(n)$ isn't divisible by 65537)

5. Choose an integer d such that:
   - The product $e \cdot d$ divided by $\Phi(n)$ has a remainder of 1, i.e., d is modular inverse of e modulo $\phi(n)$: $d \equiv e^{-1} \bmod \phi(n)$
   - in practice: we can use the Extended Euclidean algorithm to compute d

RSA Key Generation: Example

Input parameter	Value
p	7
q	19
n	$7 \cdot 19 = 133$
$\Phi(n)$	$6 \cdot 18 = 108$
e	29
d	$29^{-1} \mod 108 = 41$

RSA: Private and Public Keys

1. (n, e) will be the public key, i.e., $k_{public} = (n, e)$

2. (n, d) will be the private key, i.e., $k_{private} = (n, d)$

RSA: Encryption

The encryption function is defined as:

$$y = E_{k_{public}}(x) = x^e \mod n$$

where x is the message, y is the ciphertext, (e, n) is the public key of the person receiving the message. Indeed, only the person with the private key (d, n) can decrypt the message.

RSA: Encryption

The encryption function is defined as:

$$y = E_{k_{public}}(x) = x^e \mod n$$

where x is the message, y is the ciphertext, (e, n) is the public key of the person receiving the message. Indeed, only the person with the private key (d, n) can decrypt the message.

WARNING: As it must be, RSA uses the public key of the receiver to encrypt the message!

RSA: Decryption

The decryption function is defined as:

$$x = D_{k_{private}}(y) = y^d = (x^e)^d = x^{ed} = x^1 = x \mod n$$

where x is the message, y is the ciphertext, (d, n) is the private key of the person <u>receiving</u> the message.

WARNING: As it must be, RSA uses the private key of the receiver to decrypt the message.

RSA Encryption and Decryption: Example

Input parameter	Value
p	7
q	19
n	133
$\Phi(n)$	108
e	29
d	41

Input	Output
x	60
$y = E_{(e,n)}(x)$	$60^{29} \bmod 133 = 86$
$x = D_{(d,n)}(y)$	$86^{41} \bmod 133 = 60$

Can we encrypt with private key and decrypt with public key?

Input	Output
x	60
$y = E_{(d,n)}(x)$	$60^{41} \bmod 133 = 72$
$x = D_{(e,n)}(y)$	$72^{29} \bmod 133 = 60$

Yes, the math will work, but it's not secure. ***Why***?

Can we encrypt with private key and decrypt with public key?

Input	Output
x	60
$y = E_{(d,n)}(x)$	$60^{41} \bmod 133 = 72$
$x = D_{(e,n)}(y)$	$72^{29} \bmod 133 = 60$

Yes, the math will work, but it's not secure. ***Why***?

You cannot encrypt with your private key... because everyone has your public key... hence, everyone can decrypt your message!

Can we encrypt with private key and decrypt with public key?

Input	Output
x	60
$y = E_{(d,n)}(x)$	$60^{41} \bmod 133 = 72$
$x = D_{(e,n)}(y)$	$72^{29} \bmod 133 = 60$

Yes, the math will work, but it's not secure. ***Why***?

You cannot encrypt with your private key... because everyone has your public key... hence, everyone can decrypt your message!

NOTE: There are nonethless some use cases where encryption is done with the private key and decryption with the public key. This is not done to achieve **confidentiality**, but to achieve **authentication**. We will see this in another lecture.

RSA: Why it works?

RSA: Why it works?

NOTE: The following slides are painful. We know it. But they explain a bit the math behind RSA.

RSA: Why it works?

NOTE: The following slides are painful. We know it. But they explain a bit the math behind RSA.

To understand why it works, we need to introduce:

1. The Fermat's Little Theorem
2. The Chinese Remainder Theorem

Fermat's Little Theorem

Given a prime p, for any integer a, $a^p - a$ is a multiple of p, written as:

$$a^p \equiv a \bmod p$$

Fermat's Little Theorem

Given a prime p, for any integer a, $a^p - a$ is a multiple of p, written as:

$$a^p \equiv a \bmod p$$

Moreover if a and p are coprime then $a^{p-1} - 1$ is a multiple of p, written as:

$$a^{p-1} \equiv 1 \bmod p$$

PROOF: Check this video to see some examples that give you an intuition behind the proof of the theorem.

Chinese Remainder Theorem (CRT)

Given pairwise coprime positive integers $(n_1, n_2, \ldots, n_k)$ and an arbitrary integers $(a_1, a_2, \ldots, a_k)$, then the system:

$$\begin{aligned} x &\equiv a_1 \bmod n_1 \\ x &\equiv a_2 \bmod n_2 \\ &\vdots \\ x &\equiv a_k \bmod n_k \end{aligned}$$

has a unique solution modulo $n = n_1 \cdot n_2 \cdot \ldots \cdot n_k$.

PROOF: Check this video to see some examples that give you an intuition behind the proof of CRT.

NOTE: The practical implication of CRT is that if want to prove something when working with $x \mod n$ and $n = pq$, we can break the reasoning down into two parts: (1) $x \mod p$ and (2) $x \mod q$.

RSA: Proof of Correctness

From key generation:

$$ed \equiv 1 \bmod \Phi(n) \implies ed \equiv 1 \bmod (p-1)(q-1)$$

Thus:

$$ed \equiv 1 \bmod (p-1) \implies ed - 1 = k(p-1)$$

We prove correctness modulo (p) (similarly for (q)).

- If $\gcd(x, p) = 1$, then:

$$\begin{aligned} x^{ed} &\equiv x^{ed-1} \cdot x \equiv x^{k(p-1)} \cdot x \bmod p \\ &\equiv (x^{p-1})^k \cdot x \bmod p \\ &\equiv 1^k \cdot x \bmod p \quad \text{(Fermat's Little Theorem)} \\ &\equiv x \bmod p \end{aligned}$$

- Otherwise, it must be $\gcd(x, p) = p$ since p is prime, i.e., $x \equiv 0 \bmod p$, then:

$$x^{ed} \equiv 0 \equiv x \bmod p$$

By the CRT, $x^{ed} \equiv x \pmod{n}$.

* A more detailed proof is available in the Wikipedia **RSA article**.

RSA: Is it secure?

The main assumption behind RSA is that, given (e, n), it should be hard to compute d, p, q or $\Phi(n)$.

To make this hard:

1. p and q should two large (more than 1024 bits) distinct prime numbers
2. e should be equal or larger than 3
3. $(p-1)$ and $(q-1)$ should have large prime factors otherwise Pollard's rho algorithm is able to perform factorization efficiently

WARNING: Even when recommendations are followed, our *textbook* implementation of RSA suffers from different problems.

RSA: what if two moduli share a common factor p?

$$N_1 = p \cdot q_1$$
$$N_2 = p \cdot q_2$$
$$gcd(N_1, N_2) = p$$

Then we can easily recover q_1 and q_2:

$$q_1 = N_1/p$$
$$q_2 = N_2/p$$

RSA: avoid specific *corner case* messages

Messages such as x=0, x=1, x=(n-1) do not play well with exponentiation:

- $0^e = 0$
- $1^e = 1$
- if e is odd then: $(n-1)^2 = 1 \bmod n$, hence $(n-1)^e = (n-1) \bmod n$

RSA: small message, small exponent e

If x and e are small then:

$$y = x^e < n$$

Adversary can easily get the plaintext by computing:

$$x = \sqrt[e]{y}$$

RSA: same message encrypted with same exponent e

Given e ciphertexts of the same message x and the same exponent e is used during encryption but (p, q) different then the message x can be decrypted using the Chinese Remainder Theorem (CRT).

E.g., $e = 3$

$$\begin{aligned} y_1 &= x^3 \bmod n_1 \\ y_2 &= x^3 \bmod n_2 \\ y_3 &= x^3 \bmod n_3 \end{aligned}$$

CRT can build (here a step-by-step example):

$$y = x^3 \bmod (n_1 \cdot n_2 \cdot n_3)$$

And then $x = \sqrt[3]{y}$ since we know by construction that:

$$\begin{aligned} x &< \{n_1, n_2, n_3\} \\ x^3 &< n_1 \cdot n_2 \cdot n_3 \end{aligned}$$

RSA: same message encrypted with different coprime exponents

$$y_1 = x^{e_1} \bmod n$$
$$y_2 = x^{e_2} \bmod n$$

If we have that:

$$gcd(e_1, e_2) = 1$$

Then using EEA we can compute:

$$s \cdot e_1 + t \cdot e_2 = 1$$

But then:

$$y_1^s \cdot y_2^t = (x^{e_1})^s \cdot (x^{e_2})^t = x^{s \cdot e_1 + t \cdot e_2} = x$$

Here a step-by-step example.

RSA: small message space

Given a ciphertext y, if the message space is small (e.g., k values), then adversary can encrypt all k values and then compare results with the ciphertext y. Textbook RSA has no randomization.

RSA: How to mitigate/solve most of these attacks?

Textbook RSA is unsafe. In practice, we should always use a standard that provides rules on how to use a PK scheme for real-world usages:

> **SOLUTION**: PKCS (Public Key Cryptography Standards)

For instance:

- PKCS#1: RSA Cryptography Standard
- PKCS#3: Diffie Hellman Key Agreement Standard
- PKCS#11: Elliptic Curve Cryptography Standard

They define how to pad the messages, how to properly generate and validate the keys, etc.

How to break improper RSA?

Some tools that can be valuable:

1. **RsaCtfTool**: utility designed to decrypt data from weak public keys and attempt to recover the corresponding private key.

2. **msieve**, **yafu**, **cado-nfs**: tools for factoring large numbers.

This **blog post** provides examples (.pem files, i.e., a format for storing public/private keys) for testing known attacks.

How good we are at factorization?

- In 1991, RSA Laboratory create the RSA Factoring Challenge:
  - released 54 numbers, awarding people up to 200K for each factorization
  - the largest number successfully after 30 years has 829 bits

How good we are at factorization?

- In 1991, RSA Laboratory create the RSA Factoring Challenge:
  - released 54 numbers, awarding people up to 200K for each factorization
  - the largest number successfully after 30 years has 829 bits
- Starting from 2002, 1024-bit keys were recommended.

How good we are at factorization?

- In 1991, RSA Laboratory create the RSA Factoring Challenge:
  - released 54 numbers, awarding people up to 200K for each factorization
  - the largest number successfully after 30 years has 829 bits
- Starting from 2002, 1024-bit keys were recommended.
- Starting from 2005, 2048-bit keys were recommended.

How good we are at factorization?

- In 1991, RSA Laboratory create the RSA Factoring Challenge:
  - released 54 numbers, awarding people up to 200K for each factorization
  - the largest number successfully after 30 years has 829 bits
- Starting from 2002, 1024-bit keys were recommended.
- Starting from 2005, 2048-bit keys were recommended.
- We should be safe. However, we fear quantum computers may break 2048-bit RSA, hence the world is moving to more sophisticated asymmetric schemes, dubbed ***post-quantum cryptography***.

RSA in Python

Barebone RSA in Python [demo only!]

We can define a function to test whether a number is prime or not:

```
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

which however is extremely slow :(

We can instead use a well-known probabilistic primality test:

```
def is_prime(n, k=5):
    """Miller-Rabin primality test"""
    if n < 2 or (n > 2 and n % 2 == 0): return False
    if n == 2 or n == 3: return True

    import random
    r, d = 0, n - 1
    while d % 2 == 0:
        r, d = r + 1, d // 2

    for _ in range(k):
        a = random.randrange(2, n - 1)
        x = pow(a, d, n)
        if x == 1 or x == n - 1: continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1: break
        else:
            return False
    return True
```


* Refer to Wikipedia for more details about the **Miller-Rabin primality test**.

Then we can validate our input parameters from the example:

```
import math

# input p
p = 7
q = 19
n = p * q
phi_n = (p - 1) * (q - 1)
e = 29
d = 41

# check
assert is_prime(p), "p must be prime"
assert is_prime(q), "q must be prime"
assert p != q, "p and q must be different primes"
assert 1 < e < phi_n, "e must be in range (1, φ(n))"
assert math.gcd(e, phi_n) == 1, "e must be coprime to φ(n)"
assert (e * d) % phi_n == 1, "d must be modular inverse of e mod φ(n)"
```

Luckily, these assertions do not fail, meaning that our parameters are safe (only for our purposes!).

We perform now encryption and decryption:

```python
# input message
x = 60
print("The input message is:", x)

# encryption
y = x ** e % n
print("The encrypted message is:", y)

# decryption
z = y ** d % n
print("The decrypted message is:", z)

# this must be true!
assert x == z
```

```
The input message is: 60
The encrypted message is: 86
The decrypted message is: 60
```

However, our parameters are too small and we can factorize n:

```python
def factorize(n):
    """Trial division factorization"""
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return i, n // i
    return None, None

# Test it
p, q = factorize(133)
print(f"p = {p}, q = {q}")
```

```
p = 7, q = 19
```

The function `factorize` will not scale to large numbers.

RSA with Python [Real World Example]

We need to install the `cryptography` package:

```
pip install cryptography
```

We can now generate a key pair:

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization

# Generate a private key (2048 bits is minimum recommended, 4096 is
better)
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

# Get the public key
public_key = private_key.public_key()
```

Get the keys in a human readable format:

```
# Serialize private key to PEM format
private_pem = private_key.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.NoEncryption()
)

# Serialize public key to PEM format
public_pem = public_key.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
)

# you should save them into ".pem" files!
```

```python
print("Private key:\n", private_pem.decode())
```

```
Private key:
 -----BEGIN PRIVATE KEY-----
MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBKcwggSjAgEAAoIBAQDEvNnbaiFnfp6V
NxWS6LRop4V6wghFInIB522KS36ywgb3lAZd8zQZp5Yv1dWCk77UJuFNkZ77SK1e
W8+tib1zLu+LbBU5q43YB7QnlknBN0zswdEr1QkEMppbgxGyCUEkeOAm8GDdA68v
N7zfdWkCDx8WIA6kpLh56Xy93rNz7DpR9nG5tiJxMOlgfDTYCY4j+UmRmRDJ/Sdo
s667qeb4PxARXiiH5YYWePwIujH8AxURMO4ML0Gb1B4AsZGgiRGnhuaFfrq35rKQ
u7W2uyDjFgPI99n5vvQv1T0xB9Vj88T5tawQqYpHRJxAhgr2YKxlnYhOyEupjbez
oPpRzjydAgMBAAECggEASASTwXC32rFsrtWzxYv+4qKUR6w4p8okrtK/dBUPw7Mo
JsAeloy9QTjqTwd5eavI+tNivWj1qCniIPz96JpX6g6/7Ks7l9lXesHTLnyi8iJi
u/Z1MhTxkIJPo/k2ZJRjrsy6zTZIVw+OtPuz3Z8WugS8DfbdNWMBDsuseLrJuUCW
U2W53W/24W0JfR/kt8auKXKKTtexg0ZwMwcgeEyPEUYpHVVy+Iwff2maNcGzJ0Hx
n0w0W87oIVwZoe0F09FAhPRr2jV7xJKGBL3fF/Qm5M6nTf/GaZh3ndPg4hJcLAbV
oGxub/uk4vKovgLJEGHtaV0mYJYFaBXIxOnPvfRAFwKBgQD9U0Q9KQeczoAIDxst
N4o1ofVYYM9c1QexuSjNS3xc/liuivQMrEF7m279f2Rezq7dhmbKWyGoMyJVIXCF
wmbNKwDdLqy9iUW5Rf5KCdsf8EqThpcqGzuh/+wnIor4ErETHkopIHqxG3iV6luU
EDRfDVSl+NUz8DzgEHApHp8HhwKBgQDG0KEblpRHZhILe3CJSYEMZBuqiNa3t/xi
qO2r95dXnzzTuDsZrQILsP7N9s21PuRonXBXmvjjQQc/dnbbYYH3P5EWl0uootZH
faFtYm6ovOlZCwr9cU7w0qLkCuvL51txukaLLVOKpJzALHVaGhelPkNZRACR8Rnm
Q+CfsIObuwKBgCRkfO4oYc6WH/CIcwgI7ZMM5FlA9ylH9WYDyqfLi180rvrMPYlA
idu/Fa/XSE0Sdi6e609wziICzpDaT/b3WZtp7SLZXLmR6fFW69QpHFC3TY9qC3di
GCwKvxtULBC9ZrDvbgoAf3/ICYfH9kYMGen52UpKuB5RCf3KfsJ5grErAoGBALtv
kZQK4Hx3tGdy1uP3klx2PMcH/zZxlNYG4RuRbvNCwvvv4ngQba9MDM1s8v7+7iPa
2kEdb6wAcK7aJrApO3g4S7b6qG+yJUZ9IKWZeS5H+87g5FmQa5YLX1qTGT7BPar4
rO0lrG83Rji/MVvi4Yv+KUyNGeR9Y6d9MtYaedllAoGACpqTU3xB9DaDJd7/zxDs
```

```python
print("Public key:\n", public_pem.decode())
```

```
Public key:
 -----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAxLzZ22ohZ36elTcVkui0
aKeFesIIRSJyAedtikt+ssIG95QGXfM0GaeWL9XVgpO+1CbhTZGe+0itXlvPrYm9
cy7vi2wVOauN2Ae0J5ZJwTdM7MHRK9UJBDKaW4MRsglBJHjgJvBg3QOvLze833Vp
Ag8fFiAOpKS4eel8vd6zc+w6UfZxubYicTDpYHw02AmOI/lJkZkQyf0naLOuu6nm
+D8QEV4oh+WGFnj8CLox/AMVETDuDC9Bm9QeALGRoIkRp4bmhX66t+aykLu1trsg
4xYDyPfZ+b70L9U9MQfVY/PE+bWsEKmKR0ScQIYK9mCsZZ2ITshLqY23s6D6Uc48
nQIDAQAB
-----END PUBLIC KEY-----
```

We can now encrypt a message:

```
# Encrypt a message
message = b"Hello, RSA!"            # b means bytes
ciphertext = public_key.encrypt(
    message,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)
```

Decrypt the message:

```
# Decrypt the message
plaintext = private_key.decrypt(
    ciphertext,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)
```

```python
print(f"Original: {message}")
print(f"Encrypted: {ciphertext}")
print(f"Decrypted: {plaintext}")
assert message == plaintext
```

```
Original: b'Hello, RSA!'
Encrypted:
b'\x03\xdc\x84\x06\x86V\x1bK+\xf4\xa7\xb9>C\x1a`\xc5\x9e\xfb\x9b"*9\xeb\x
\x85\x9229+\xf2,\\\xb3\xaa\\\xc4\x96\xf9hY\r\xa0\xc7\xde\x8eM\x108u\xe2\x
Decrypted: b'Hello, RSA!'
```

Hybrid Cryptography

RSA cannot encrypt large messages!

We have three major issues with RSA:

- the message to be encrypted must be smaller than the modulus
- the encryption, based on exponentiation, is slow
- we cannot make the modulus too large, otherwise the encryption would be too slow

RSA cannot encrypt large messages!

We have three major issues with RSA:

- the message to be encrypted must be smaller than the modulus
- the encryption, based on exponentiation, is slow
- we cannot make the modulus too large, otherwise the encryption would be too slow

WARNING: Some implementations of RSA may work around these issues, e.g., by splitting the message into chunks. But this is not a robust solution; the underlying issues are still there. There is no standard for message chunking in RSA since the longer the message, the slower the encryption/decryption process.

RSA cannot encrypt large messages!

We have three major issues with RSA:

- the message to be encrypted must be smaller than the modulus
- the encryption, based on exponentiation, is slow
- we cannot make the modulus too large, otherwise the encryption would be too slow

WARNING: Some implementations of RSA may work around these issues, e.g., by splitting the message into chunks. But this is not a robust solution; the underlying issues are still there. There is no standard for message chunking in RSA since the longer the message, the slower the encryption/decryption process.

IMPORTANT: we typically cannot encrypt large messages with asymmetric cryptography.

We can observe this when using a real-world implementation of RSA:

```python
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
public_key = private_key.public_key()

large_message = b"A" * 1000  # Too large!

try:
    ciphertext = public_key.encrypt(
        large_message,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
except ValueError as e:
    print(f"Error: {e}")  # Message too long for RSA encryption
```

```
Error: Encryption failed
```

Meet Hybrid Cryptography

In the real world, we combine the best of both worlds:

- we use asymmetric cryptography for key exchange, at the beginning of the conversation, where we need to exchange a few messages, which are very short.
- we use symmetric cryptography for data encryption for the rest of the conversation, where we need to exchange a lot of messages, which could even be very long.

This is called ***hybrid cryptography***.

NOTE: We will see this in action when considering the Public Key Infrastructure (PKI).